Assignment #2 Overview

Design and Analysis of Algorithms

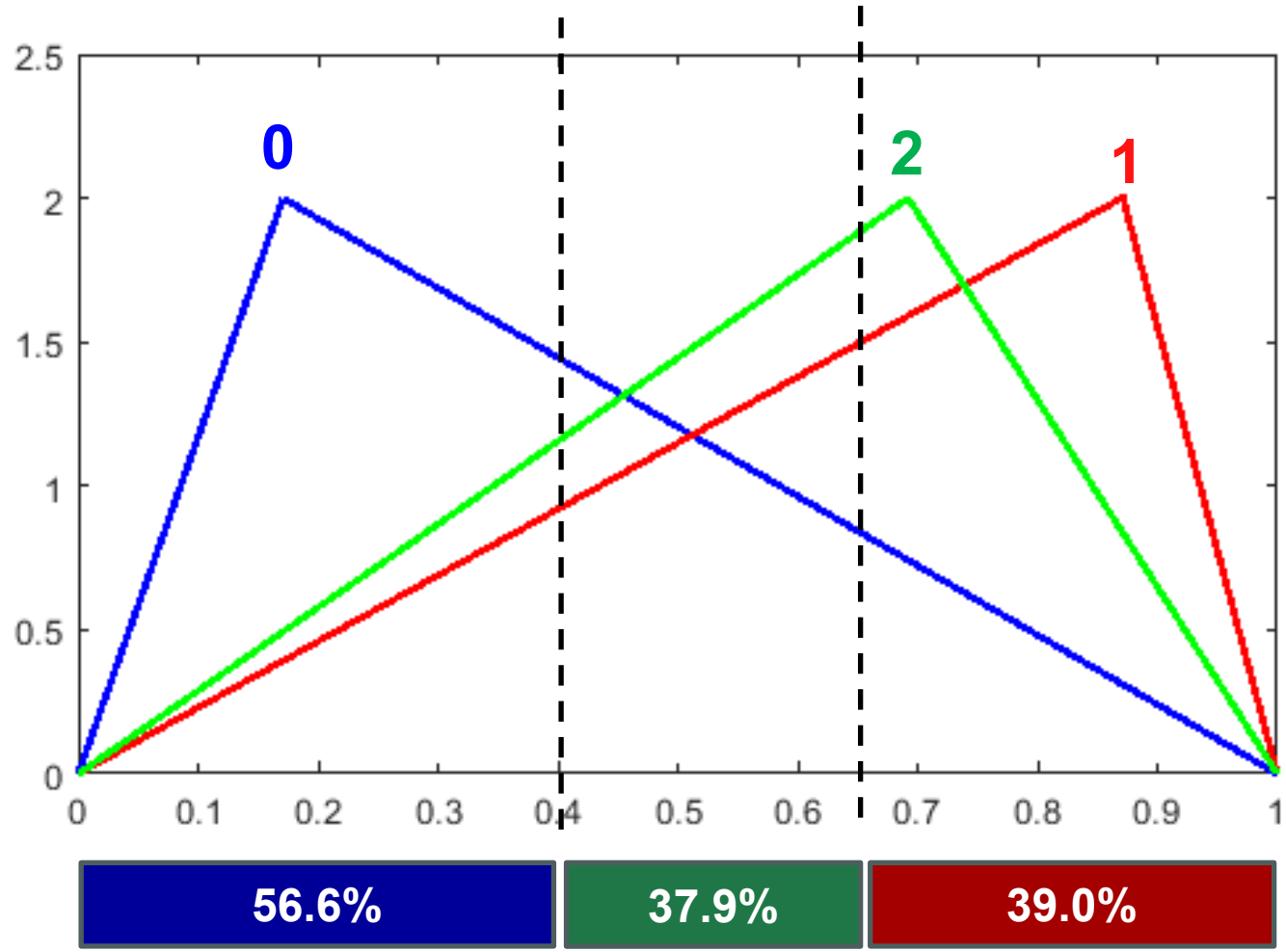
Brian C. Dean

School of Computing

Clemson University

Assignment #2: Fair Division

- Core type of problem in algorithmic game theory.
- Subdivide unit interval [0,1] among N players, so each feels that they are getting ≥ 1/N of its value.
- Different players have different “value density” functions.



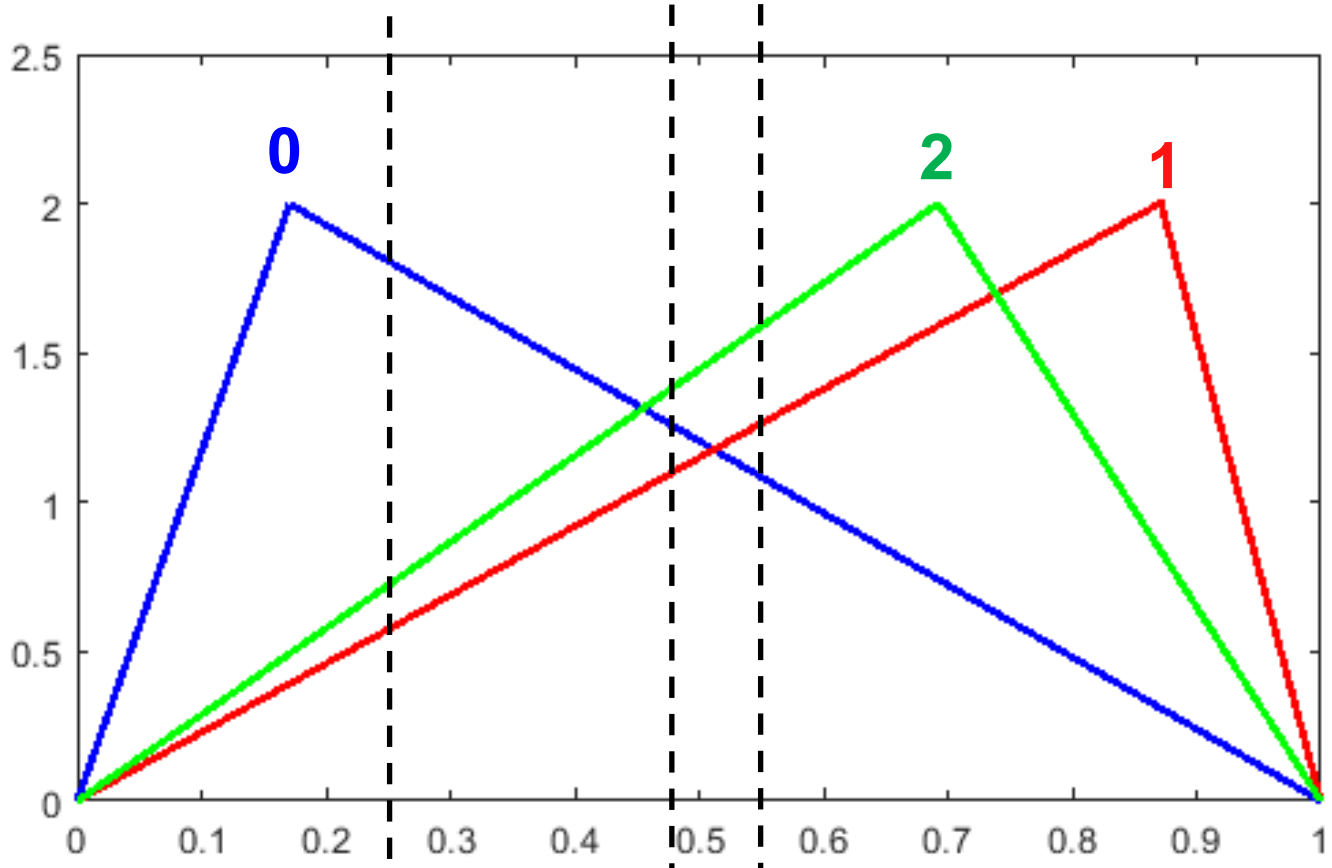
Classic Problem: Fairly Cutting a Cake

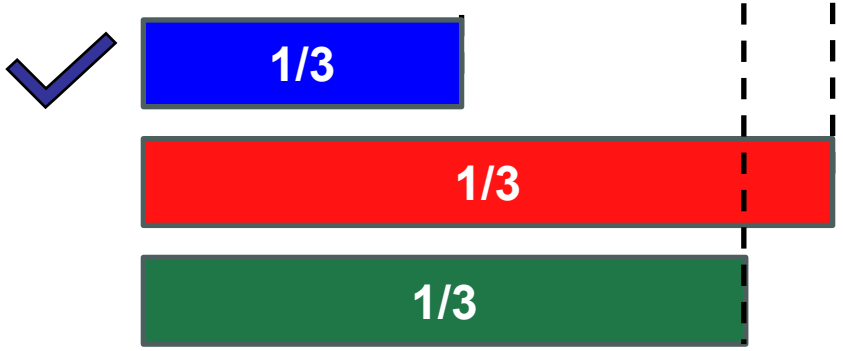
Between two players:

(I) *Player 1 makes a cut and player 2 chooses the side of the cut.*

(II) *Slowly move a knife across the cake, stopping when one player says "stop!", claiming that piece.*

Between N Players…





Find “1/N” cutoff for each player using their value density function.

Player with earliest cutoff gets that slice (making them happy).

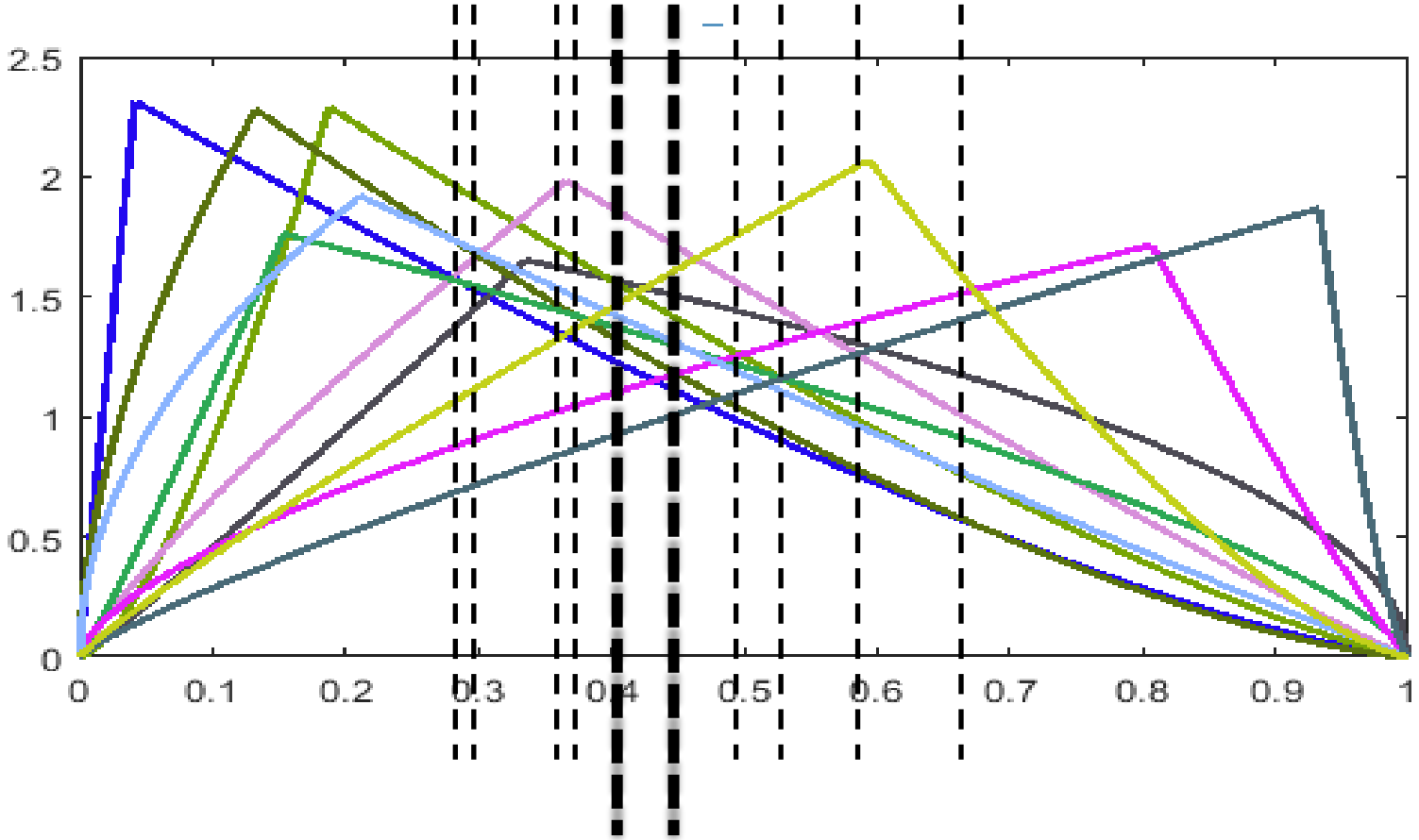
Remaining players still have at least $1 - 1/N = (N - 1) / N$ value in play. If subdivided properly, they’ll each still get at least

$$1 / (N - 1) \times (N - 1) / N = 1 / N$$

worth of value.

Bad news: $\Theta(N^2)$ total time…

Divide and Conquer!



Subproblem 1
N / 2 players
(each allocated ≥ ½ their value)

Subproblem 2
N / 2 players
(each allocated ≥ ½ their value)

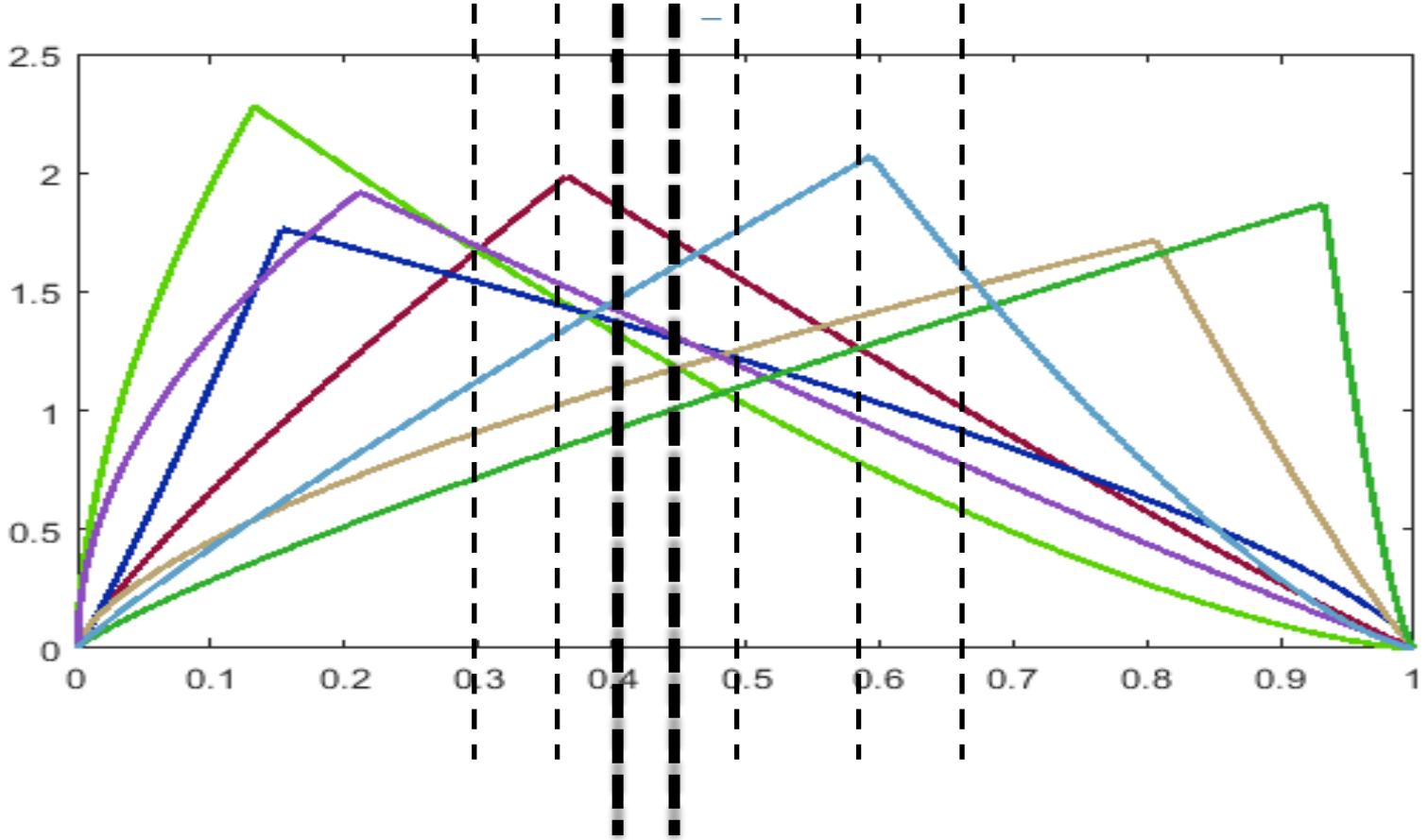
If N is even:

- Find "1/2" cutoff for each player.
- Split on median of those cutoffs (two possibilities).
- Recurse on two subproblems:
  $T(n) = 2T(n / 2) + \Theta(n)$

If N is odd:

- Do one step of the prior algorithm to assign one slice, making N even.

Divide and Conquer!



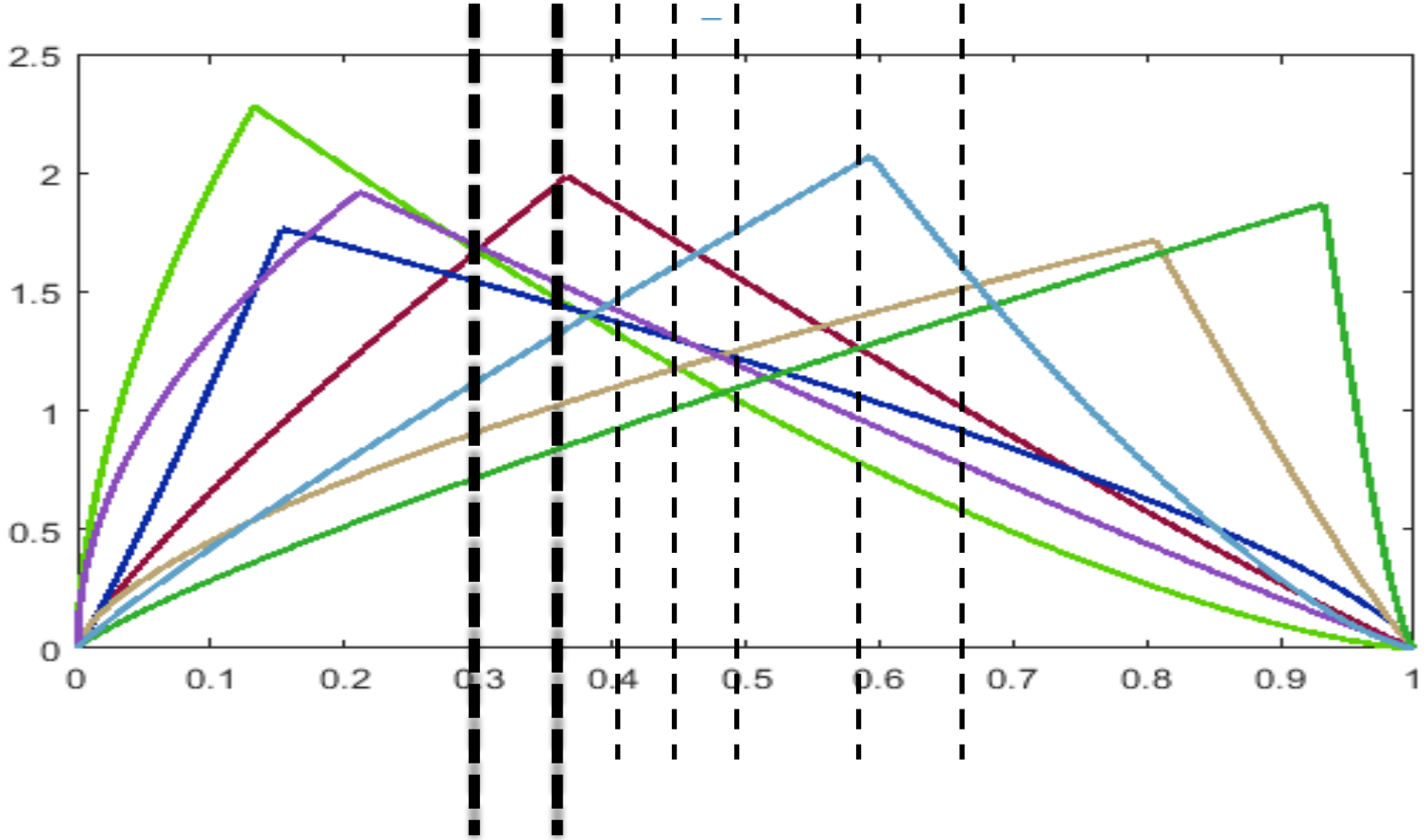
Subprob 1
⌊N/2⌋ players
(allocated ≥ ⌊N/2⌋ / N value)

Subproblem 2
⌈N/2⌉ players
(allocated ≥ ⌈N/2⌉ / N value)

We can also handle the case where N is odd with a more balanced split.

Instead of “1/2”, we’d use ⌊N/2⌋ / N as the cutoff point.

Divide and Conquer!

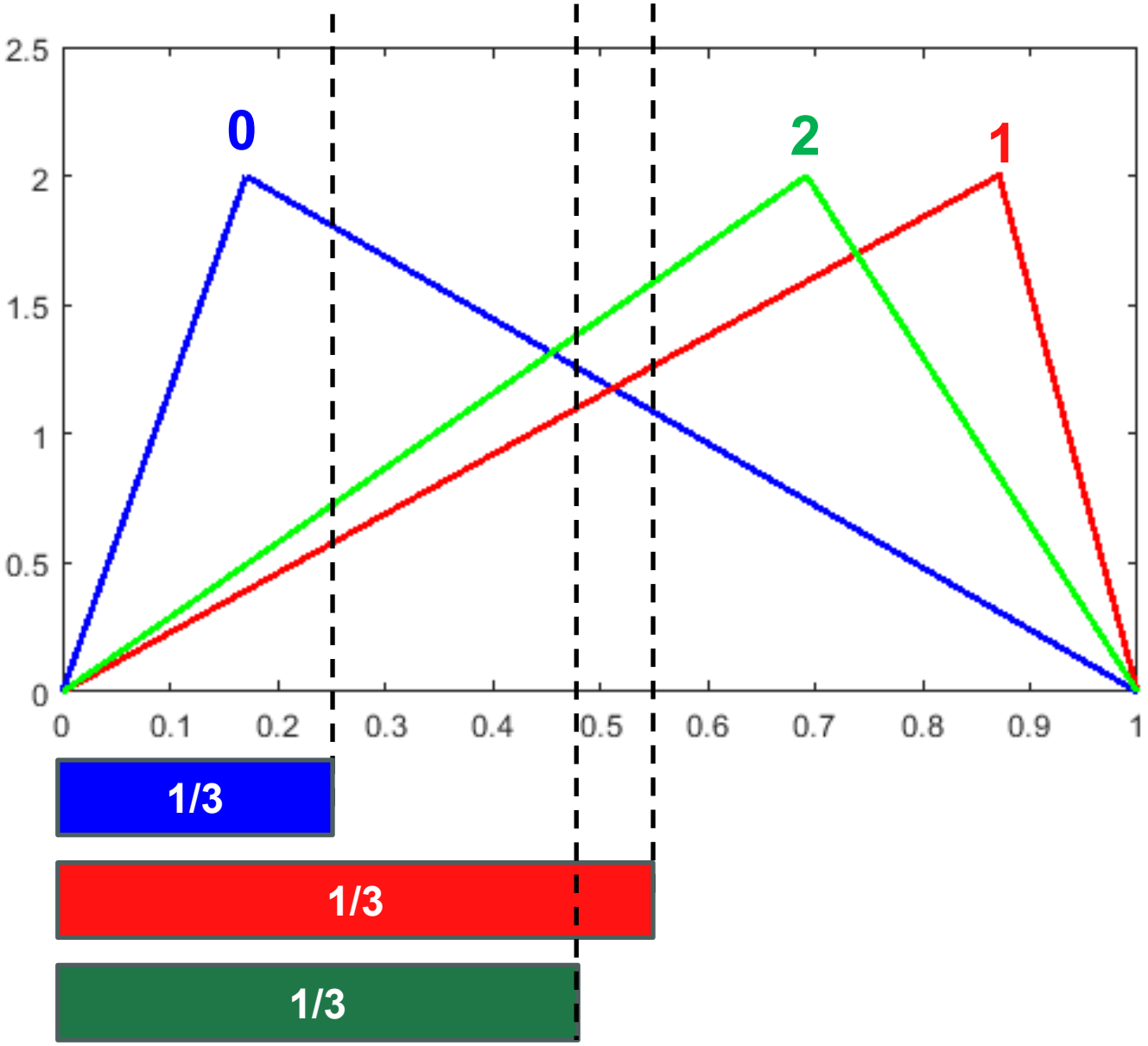


Subprob 1
1 player
(allocated ≥ 1/N value)

Subproblem 2
N – 1 players
(each allocated ≥ 1 – 1/N their value)

The “1 slice” case can also be viewed with the same divide and conquer mechanism – as less of a special case.

Program Structure



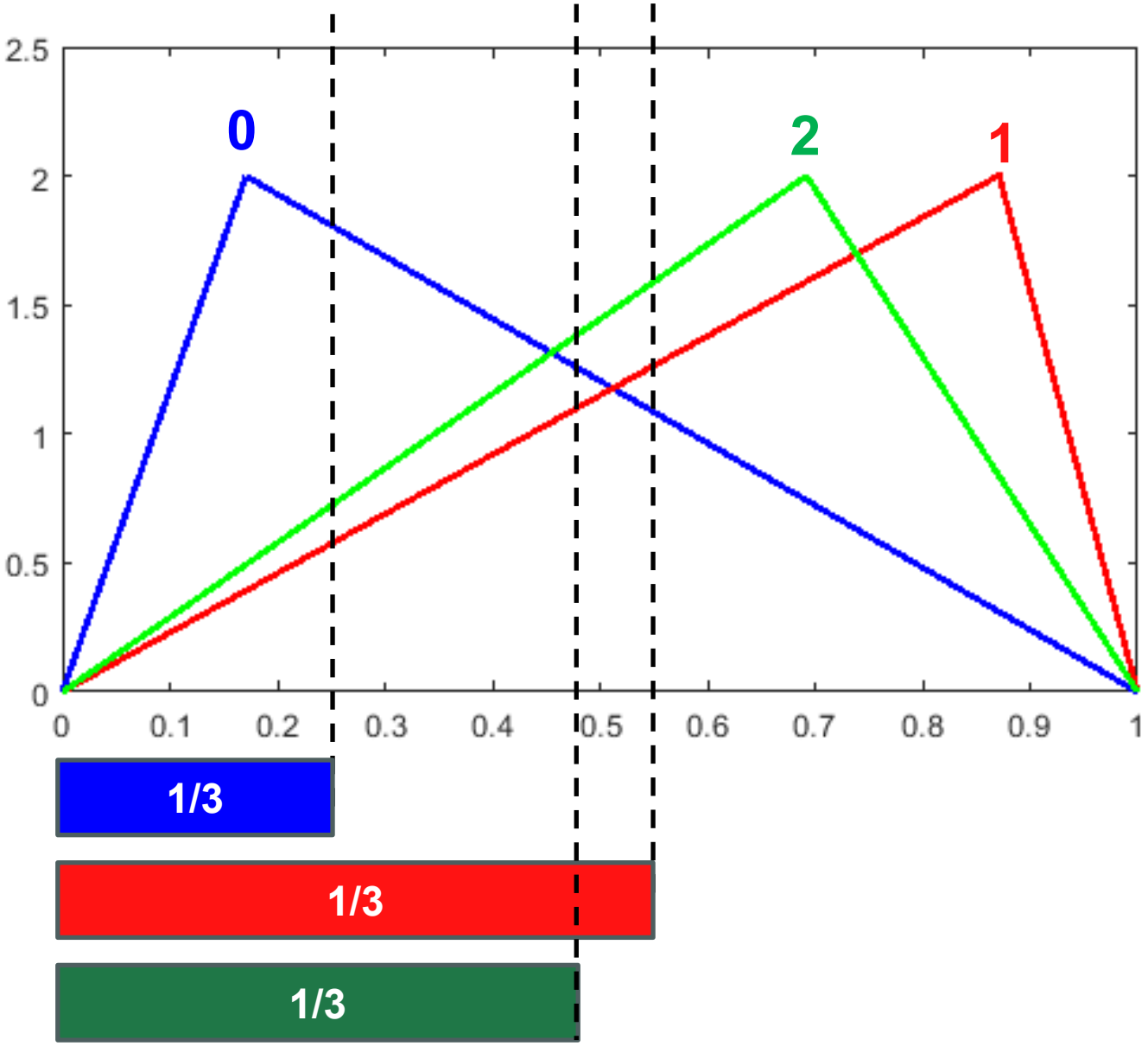
N ≤ 250,000 (so $\Theta(N^2)$ solutions will time out on large cases)

Your program can call "compare" ("compare_local") to compare cut points. E.g.,

compare(**2**, **0**, 0.333) → +1
compare(**2**, **1**, 0.333) → –1

At the end, print out the order of the segments allocated to the players: **0 2 1**

Program Structure

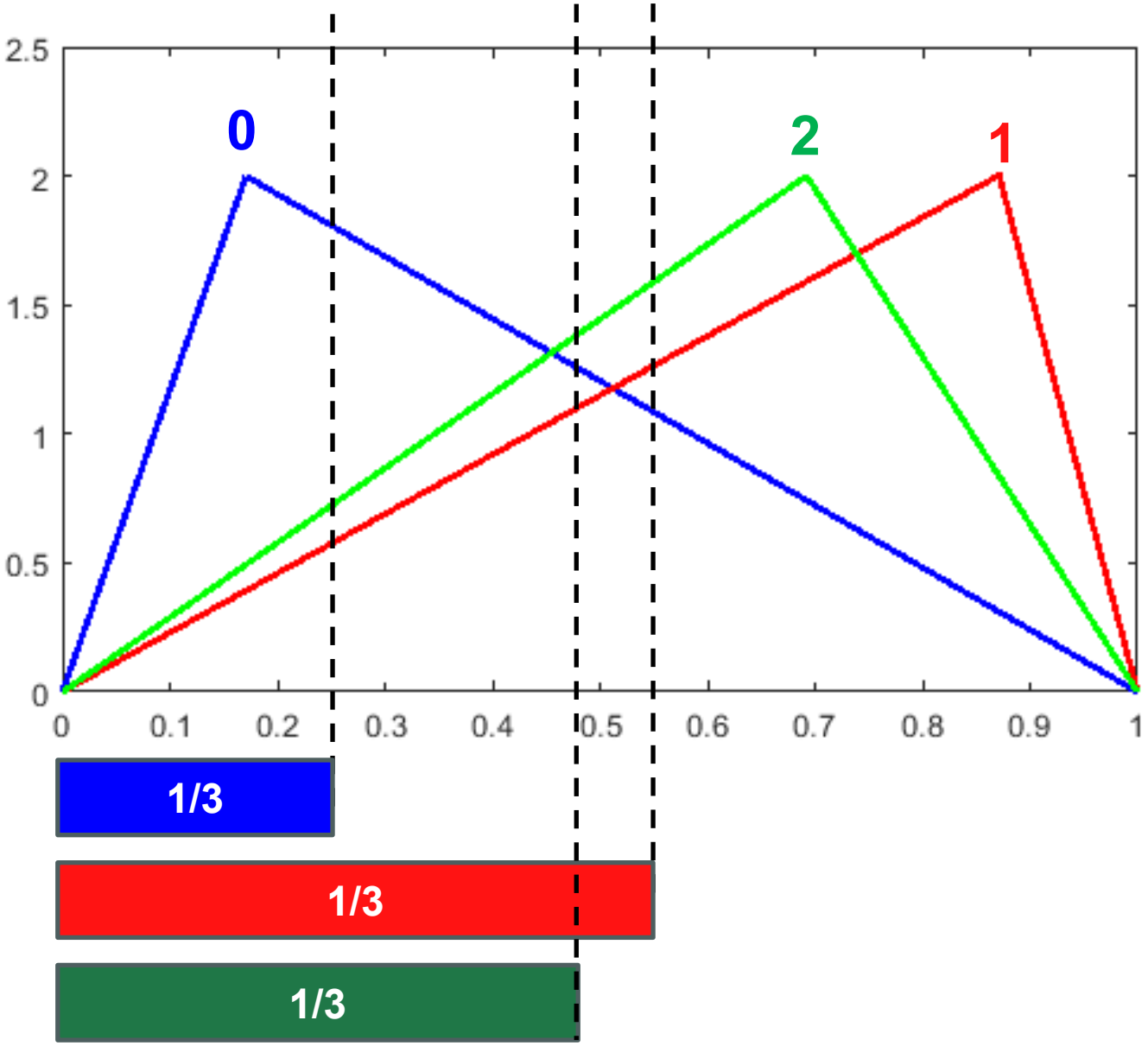


At the end, print out the order of the segments allocated to the players: **0 2 1**

The grader will use your ordering to try and allocate as much value as possible uniformly to everyone (you want this to be at least 1/N; you'll likely be able to get even more).

Your score on each test case is based on the value everyone gets.

Divide and Conquer^2

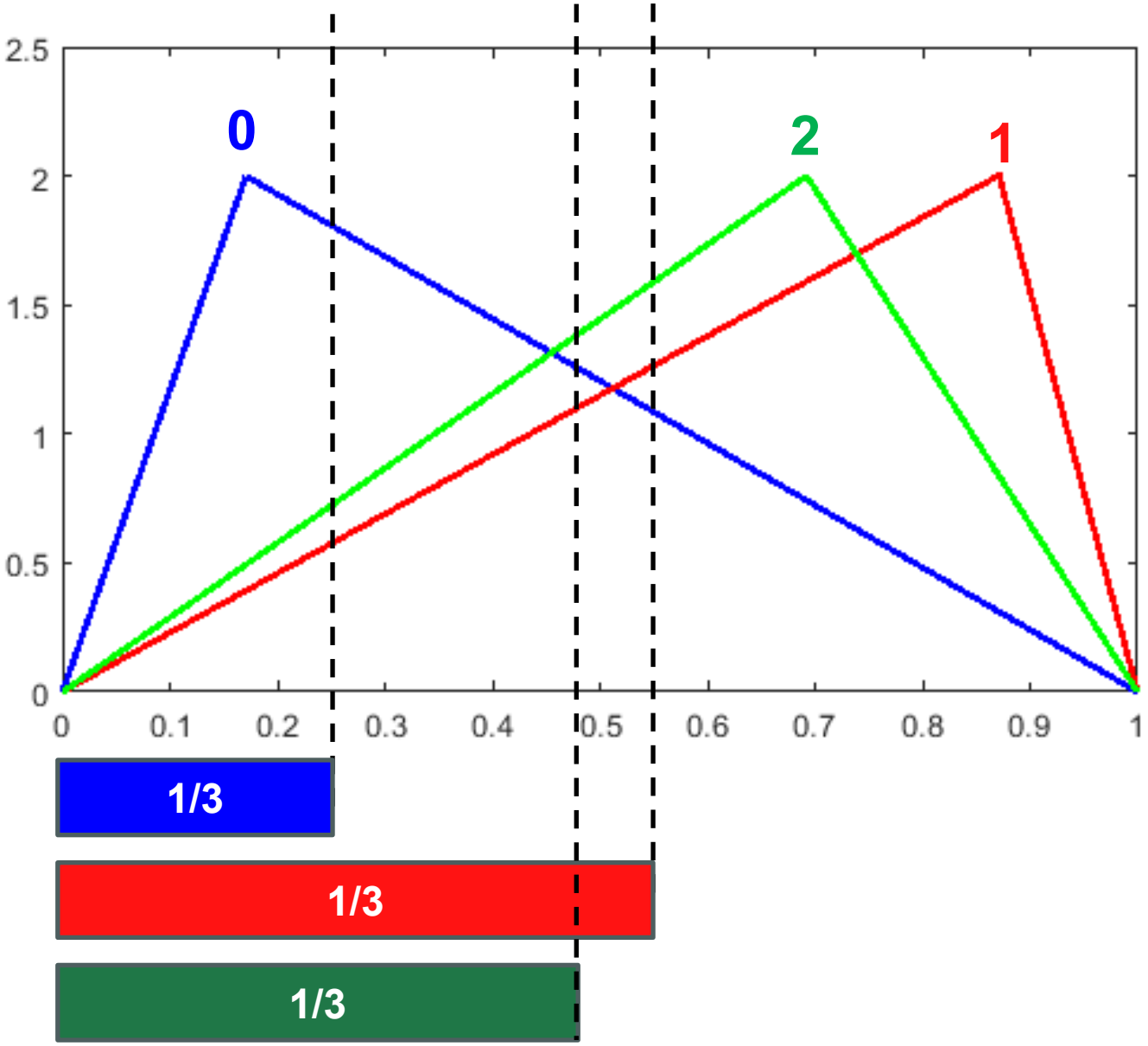


Two divide and conquer algorithms to implement:

1. The overall fair division algorithm.
2. Quickselect, to choose the dividing point at each step.

The grader will monitor the comparisons you make; penalty applied to your score if it deems you aren't doing #2 properly (e.g., if you are just sorting).

Recommended Approach



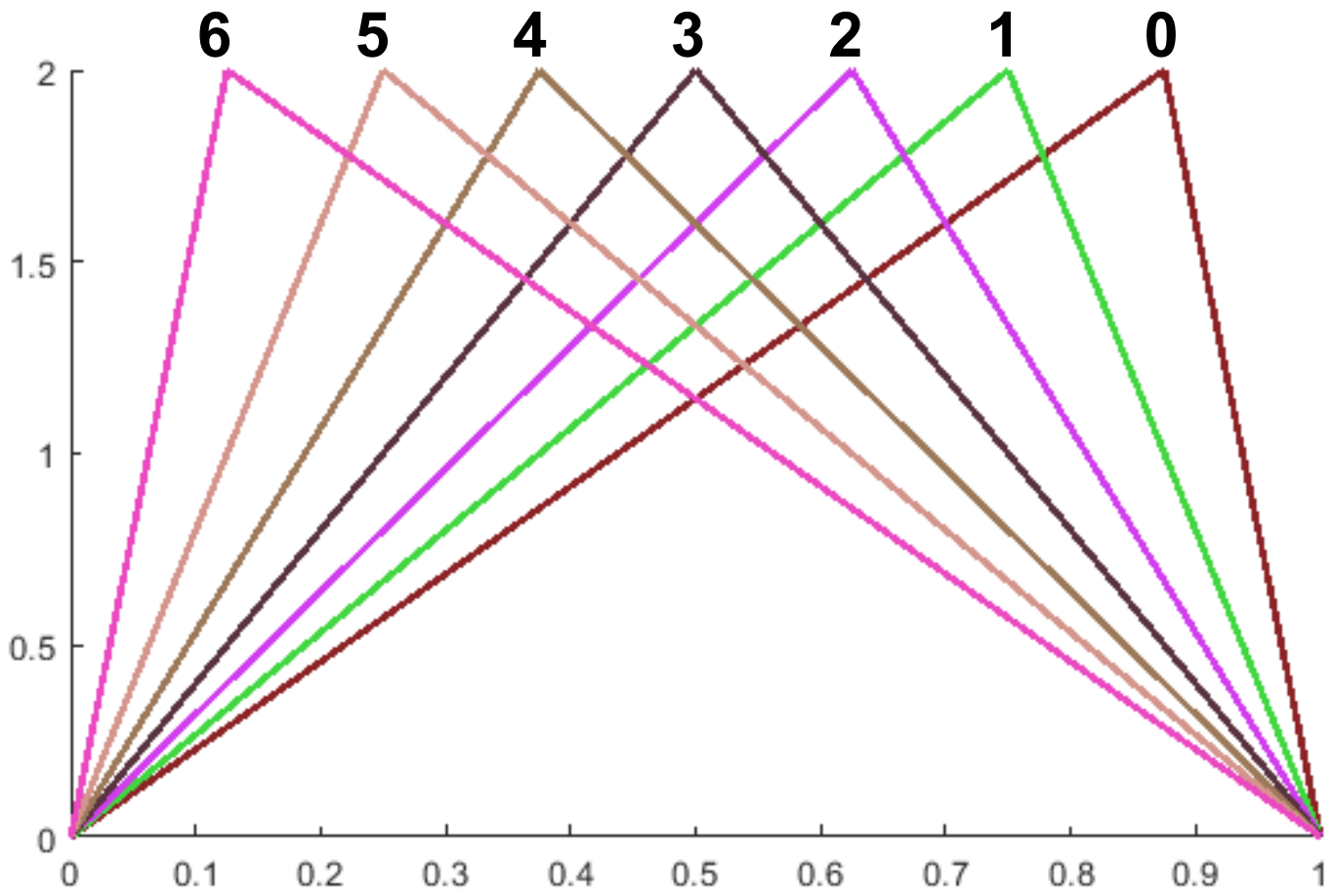
Recommended to try first:

1. The overall fair division algorithm.
2. Built-in sort, to find the dividing point at each step of #1.

This will give a penalty on your score at the moment, but can help ensure your algorithm for #1 is correct before adding quickselect.

(Note: $T(N) = 2T(N/2) + O(N \log N)$ also yields a slightly slower running time of $O(N \log^2 N)$).

Local Testing



The compare_local() function lets you test your code locally for any value of N you want.

It models players' value density functions as triangles with uniformly-spaced peaks, in order N – 1 … 0 (so this order of players is the best answer).